Matrix Profile for Time Series Motifs and Discords

CEFET/RJ

Data Analytics Lab

dal.global@grupo.cefet-rj.br | https://eic.cefet-rj.br/dal

Why motifs matter

Motifs reveal recurring patterns that help explain the structure of a time series — and appear in critical real-world domains.



Health

Patterns in ECG, EEG, and physiological signals reveal recurring clinical states.



Climate & Sensors

Climate cycles and sensor failures appear as repeated motifs.



Finance

Recurring price and volume patterns can anticipate market behavior.

> 💡 The absence of an expected motif can also signal anomalous behavior.

Motif discovery vs anomaly detection

🔍 Motif Discovery

- Seeks repetition: similar subsequences that appear more than once.
- Result: set of recurring patterns with support ≥ σ.
- Example: normal heartbeat that repeats throughout the ECG.

⚠️ Anomaly Detection (Discords)

- Seeks rarity or deviation: unusual or isolated subsequences.
- Result: subsequences with no close neighbors (high value in M).
- Example: arrhythmia that does not fit any recurring pattern.

> Both problems are connected: what is not a motif may be a discord.

Motif: definition, occurrence, and formalization

- A motif is a recurrent subsequence in a time series.
- The subsequence does not need to be known in advance — it is discovered by scanning the series.
- We extract from X all sliding windows W of size p.
- The occurrences of q are the windows sufficiently similar to q (distance ≤ ε or similarity ≥ threshold).
- q is a motif when the number of occurrences reaches the minimum support σ.

p = word size; ε = distance threshold; σ = minimum support; W = set of windows

Formalization

$$occurs(q, X) = \{w_i \mid w_i \in W \ \wedge \ dist(w_i, q) \leq \varepsilon\}$$

$$motif(q, X) \Leftrightarrow |occurs(q, X)| \geq \sigma$$

> The **exclusion zone** prevents counting overlapping neighboring windows as distinct occurrences: |i − j| > ez, with ez = p/2.

ez = exclusion zone; |i − j| = temporal distance between windows

Motif Discovery Taxonomy

Taxonomy Dimensions

Representation

How the series is reduced or encoded before the search. Examples: PAA (aggregation), SAX (symbolic), z-score (normalization).

Similarity

Which function measures proximity between subsequences. Examples: Euclidean (lock-step), DTW (elastic), Hamming (symbolic), MINDIST (SAX lower bound).

Search Method

How candidates are generated and verified. Examples: brute-force (exact), index-based (PAA/SAX), random projection (probabilistic), Matrix Profile (structural).

Discovery depends on indexing, similarity function, and search method.

Discovery methods: overview

- Comparing all subsequences directly is expensive: **$O(n^2)$ pairs**.
- Indexing reduces dimensionality, discretizes values, or organizes candidates.
- The goal is to speed up search while preserving relevant patterns.
- The choice of method depends on series size, distance function, and desired support.

Brute-force

Compares everything with everything. Exact, but $O(n^2 \cdot p)$. Correctness baseline.

Index-based

Reduces and discretizes via PAA/SAX. Groups candidates symbolically.

Random Projection

Uses sampling and collisions to identify probable candidates.

Matrix Profile

Structures all nearest-neighbor distances. Reveals motifs and discords in a single pass.

Normalization: scale and methods

- Two subsequences can have the same shape at different scales.
- Without normalization, amplitude and offset dominate the distance.
- Normalization makes comparison focused on the shape of the pattern.
- Z-score is the standard in motif discovery: produces zero mean and unit variance.

$$z(x_t) = \frac{x_t - \mu}{\sigma}$$

μ = window mean; σ = window standard deviation

Z-Score

Produces zero mean and unit variance. Standard in motif discovery and Matrix Profile.

Min-Max

Scales values to [0, 1]. Useful when relative amplitude matters.

Adaptive normalizations

Handle local non-stationarity better. Applied window by window rather than globally.

PAA: intuition and formalization

- Reduces the series by averaging equal-sized segments.
- The signal becomes more compact and less sensitive to local noise.
- It is a natural bridge to symbolic methods such as SAX.

$$x'_j = \frac{n}{n'} \sum_{i=\frac{n}{n'}(j-1)+1}^{\frac{n}{n'}j} x_i$$

n = original length; n' = number of segments; p = word size

```
INPUT: time series X of length n, number of segments n'
PRECONDITION: n divisible by n'; X normalized (z-score)

STEP 1 — Compute segment size: seg = n / n'
STEP 2 — For j from 1 to n':
   start = (j-1) * seg + 1
   end   = j * seg
   x'[j] = mean(X[start .. end])
STEP 3 — Return X' = [x'[1], x'[2], ..., x'[n']]

OUTPUT: reduced series X' of length n'
POSTCONDITION: each x'[j] represents the mean of the j-th segment of X
```

SAX: Symbolic Representation

- Transforms numerical values into symbols from an alphabet of size α.
- Breakpoints β divide the normal distribution into α equiprobable regions.
- Each PAA segment is assigned the symbol of the region it falls into.
- Windows of size p become symbolic words of length n'.

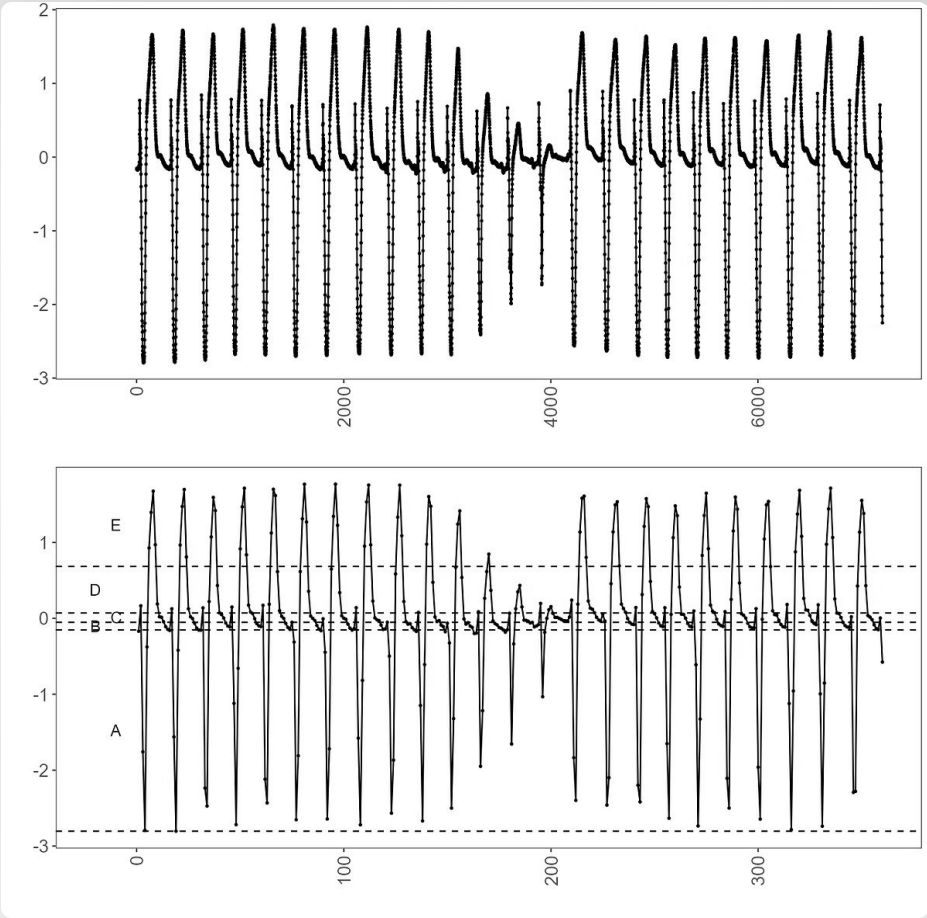
> **$x_t \in [\beta_{k-1}, \beta_k] \rightarrow$ symbol a_k**

α = alphabet size; β = breakpoints; n' = word size

```
INPUT: series X of size n, word size n', alphabet α
PRECONDITION: X normalized (z-score); breakpoints β pre-computed

STEP 1 — Apply PAA: X' = PAA(X, n')
STEP 2 — For j from 1 to n':
   For k from 1 to α:
      If x'[j] ∈ [β[k-1], β[k]]:
         symbol[j] = a_k
         stop search
STEP 3 — Word = concatenate(symbol[1..n'])
STEP 4 — Repeat for each sliding window of X

OUTPUT: sequence of symbolic words
POSTCONDITION: each word represents a window of size p of X
```



SAX: properties and MINDIST distance

- SAX preserves neighborhood structure: nearby words correspond to nearby subsequences.
- MINDIST is a lower bound of the Euclidean distance between original subsequences.
- Allows discarding candidates without computing the full numerical distance.
- The larger the alphabet α, the smaller the approximation error.

$$MINDIST(\hat{q}, \hat{q}') = \sqrt{\frac{p}{n'}} \sum_{i=1}^{n'} dist(\hat{q}_i, \hat{q}'_i)$$

p = window size; n' = word size; dist = lookup table of distances between symbols

Symbolic distance table (α = 4)

	a	b	c	d
a	0	0	0.67	1.34
b	0	0	0	0.67
c	0.67	0	0	0
d	1.34	0.67	0	0

Values based on the breakpoints of the standard normal distribution for α=4.

Similarity: measures and choice

- Motif discovery depends on deciding when two subsequences are similar.
- The choice of distance measure changes which patterns will be considered motifs.
- There is no universal measure: the choice depends on the phenomenon and expected distortions.
- Lock-step measures compare point by point with rigid alignment.
- Elastic measures allow temporal misalignment and rhythm variations.

Lock-step

Compares corresponding positions directly. Simple and efficient. Examples: Euclidean, Hamming. Sensitive to temporal shifts.

Elastic

Allows aligning points out of order. Captures shape similarity even with rhythm variations. Classic example: DTW.

> "The choice between lock-step and elastic is one of the most impactful decisions in the motif discovery pipeline."

Euclidean Distance

- Compares two sequences of the same size point by point.
- Simple, fast, and widely used in motif discovery.
- Works well when subsequences are aligned in time.
- Sensitive to amplitude differences — requires prior normalization.

$$EDist(q, q') = \sqrt{\sum_{i=1}^{p} (q_i - q'_i)^2}$$

p = sequence length; qi, q'i = values at corresponding positions

Properties

Complexity

O(p) per pair of subsequences. Supports early-abandon for efficient pruning.

Lower bound

LB_Keogh and LB_Kim are lower bounds of the Euclidean distance used for pruning in indexed searches.

Limitation

Does not tolerate temporal shifts. Two sequences with the same shape but out of phase will have a high distance.

Hamming Distance

- Counts differing positions between symbolic sequences of the same size.
- Natural for SAX words: measures how many letters differ.
- Measures how many substitutions would be needed to make the sequences equal.
- Used to group motif candidates in the index-based pipeline.

$$HDist(q, q') = \sum_{i=1}^{p} HItem(q_i, q'_i)$$

$$HItem(x, y) = \begin{cases} 0 & \text{if } x = y \\ 1 & \text{otherwise} \end{cases}$$

p = word size; *qi*, *q'i* = symbols at corresponding positions

Properties and Usage

Complexity

O(p) per pair. Very efficient for short SAX words.

Grouping

Windows with HDist ≤ h are candidates for the same motif. Controls the trade-off between recall and precision.

Limitation

Ignores the magnitude of the difference between symbols. MINDIST corrects this using the symbolic distance lookup table.

DTW: Intuition and Motivation

- Dynamic Time Warping allows aligning sequences that evolve at different rhythms.
- The method searches for a least-cost path in an alignment matrix.
- It is useful when shape matters more than exact temporal alignment.
- Each point of one sequence can be aligned to one or more points of the other.
- The Sakoe-Chiba constraint (band w) limits the maximum alignment deviation.

Build cost matrix

Each cell (i, j) receives the cost of aligning q[i] with q'[j].

Find optimal path

Dynamic programming finds the least-cost path from cell (1,1) to (p,p).

Calculate distance

The DTW distance is the square root of the accumulated cost along the optimal path.

DTW is more robust than Euclidean distance for series with rhythm variations, but has $O(p^2)$ cost per pair.

DTW: dynamic programming

- The matrix accumulates the minimum cost to align prefixes of the two sequences.
- Each cell depends on three already-computed neighbors.
- The final cost is the elastic distance between the sequences.
- The Sakoe-Chiba constraint (band w) limits the maximum alignment deviation.

$$dtw_{i,j} = cost(q_i, q'_j) + \min(dtw_{i-1,j},\ dtw_{i,j-1},\ dtw_{i-1,j-1})$$

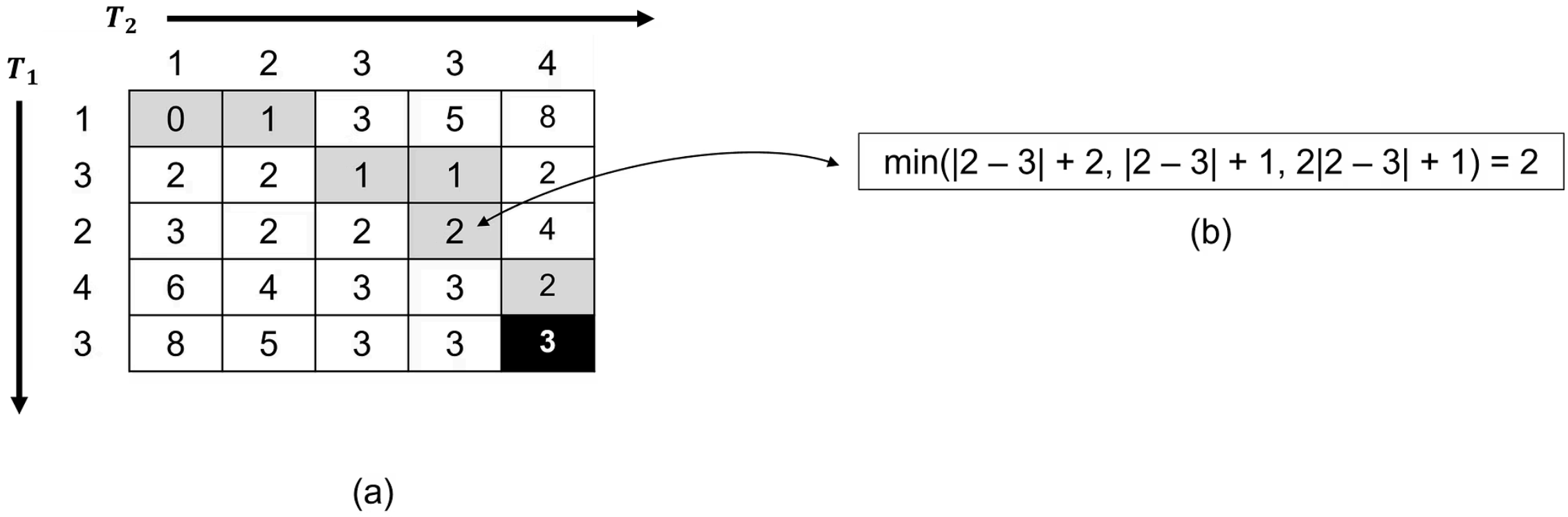
p = sequence length; w = constraint band width

```
INPUT: sequences q and q' of length p, band w
PRECONDITION: q and q' normalized; w ≥ 0

STEP 1 — Initialize matrix DTW[0..p][0..p] with ∞
         DTW[0][0] = 0
STEP 2 — For i from 1 to p:
    For j from max(1, i-w) to min(p, i+w):
        cost = (q[i] - q'[j])²
        DTW[i][j] = cost +
            min(DTW[i-1][j], DTW[i][j-1], DTW[i-1][j-1])
STEP 3 — Return sqrt(DTW[p][p])

OUTPUT: DTW distance between q and q'
POSTCONDITION: value ≥ 0; equals zero if q = q'
```

Visual Example of DTW



DTW between two small sequences, showing the cost matrix and calculation via dynamic programming.

Lower bounds and pruning

Lower bounds

Estimate a lower bound of the actual distance. If the bound already exceeds ε, the pair is discarded without computing the full distance. Examples: LB_Keogh (DTW), LB_Yi, LB_Kim.

Early-abandon

Stops the point-by-point computation as soon as the accumulated value exceeds ε^2. Compatible with Euclidean distance and DTW.

Index-based pruning

Symbolic indices (SAX) or data structures (iSAX, R-tree) eliminate entire regions of the search space before any distance computation.

> Combining lower bounds with early-abandon can reduce practical cost by orders of magnitude.

Brute-force discovery: intuition

Brute-force discovery

- Extracts all subsequences of size *p* from the series.
- Compares each subsequence with all others (except exclusion zone).
- Counts occurrences: windows with distance below ε.
- Exact, but complexity $O(n^2)$ times the cost of the distance.

p = word size; ε = distance threshold; σ = minimum support; ez = exclusion zone

```
INPUT: series X of size n, size p, threshold ε, support σ
PRE-CONDITION: X normalized; ez = p/2 (default exclusion zone)

STEP 1 — W = extract all windows of X of size p
STEP 2 — motifs = {}
STEP 3 — For each w_i in W:
    occurrences = 0
    For each w_j in W, |i - j| > ez:
        If dist(w_i, w_j) ≤ ε:
            occurrences += 1
    If occurrences ≥ σ:
        motifs.add(w_i)
STEP 4 — Return motifs
OUTPUT: set of motif subsequences
POST-CONDITION: each motif occurs ≥ σ times with distance ≤ ε
```

Brute-force: algorithm & complexity

Brute-force: complexity & optimizations

- Total complexity: $O(n^2 \cdot p)$ for Euclidean distance.
- With DTW, the cost per pair is $O(p^2)$, raising to $O(n^2 \cdot p^2)$.
- Early-abandon reduces practical cost without affecting exactness.
- Prior normalization is essential for meaningful comparisons.

n = series length; *p* = window size

```
INPUT: sequences w_i and w_j of size p, threshold ε
PRE-CONDITION: w_i and w_j normalized

STEP 1 — accumulated = 0
STEP 2 — For t from 1 to p:
    accumulated += (w_i[t] - w_j[t])²
    If accumulated > ε²:
        Return ∞   (early abandon)
STEP 3 — Return sqrt(accumulated)

OUTPUT: Euclidean distance or ∞ if threshold exceeded
POST-CONDITION: result ∞ discards the pair without full cost
```

Index-based discovery: PAA/SAX

Theory

- Avoids directly comparing all numerical subsequences.
- Uses PAA and SAX to represent windows as symbolic words.
- Windows with the same word (or similar words) are motif candidates.
- Groups with sufficient support are verified numerically.

p = word size; *α* = alphabet; *σ* = minimum support; *ε* = verification threshold

Pseudocode

```
INPUT: series X, size p, alphabet α, support σ, threshold ε
PRE-CONDITION: X normalized (z-score)

STEP 1 — W = extract sliding windows from X of size p
STEP 2 — For each w_i in W:
   X'_i = PAA(w_i, n')
   word_i = SAX(X'_i, α)
STEP 3 — Group windows by symbolic word
STEP 4 — For each group with |group| ≥ σ:
   Verify numerically: dist(w_i, w_j) ≤ ε
   If confirmed: register as motif candidate
STEP 5 — Return confirmed motifs

OUTPUT: set of motifs with support ≥ σ
POST-CONDITION: false positives eliminated in numerical verification
```

Index-based: grouping and verification

Index-based: grouping and verification

- Symbolic windows that are equal or close (Hamming distance ≤ h) are grouped together.
- Groups with sufficient support (|group| ≥ σ) are motif candidates.
- Numerical verification eliminates false positives introduced by discretization.
- The symbolic index can be a hash table or trie over SAX words.

h = Hamming threshold; *σ* = minimum support; *ε* = numerical verification threshold

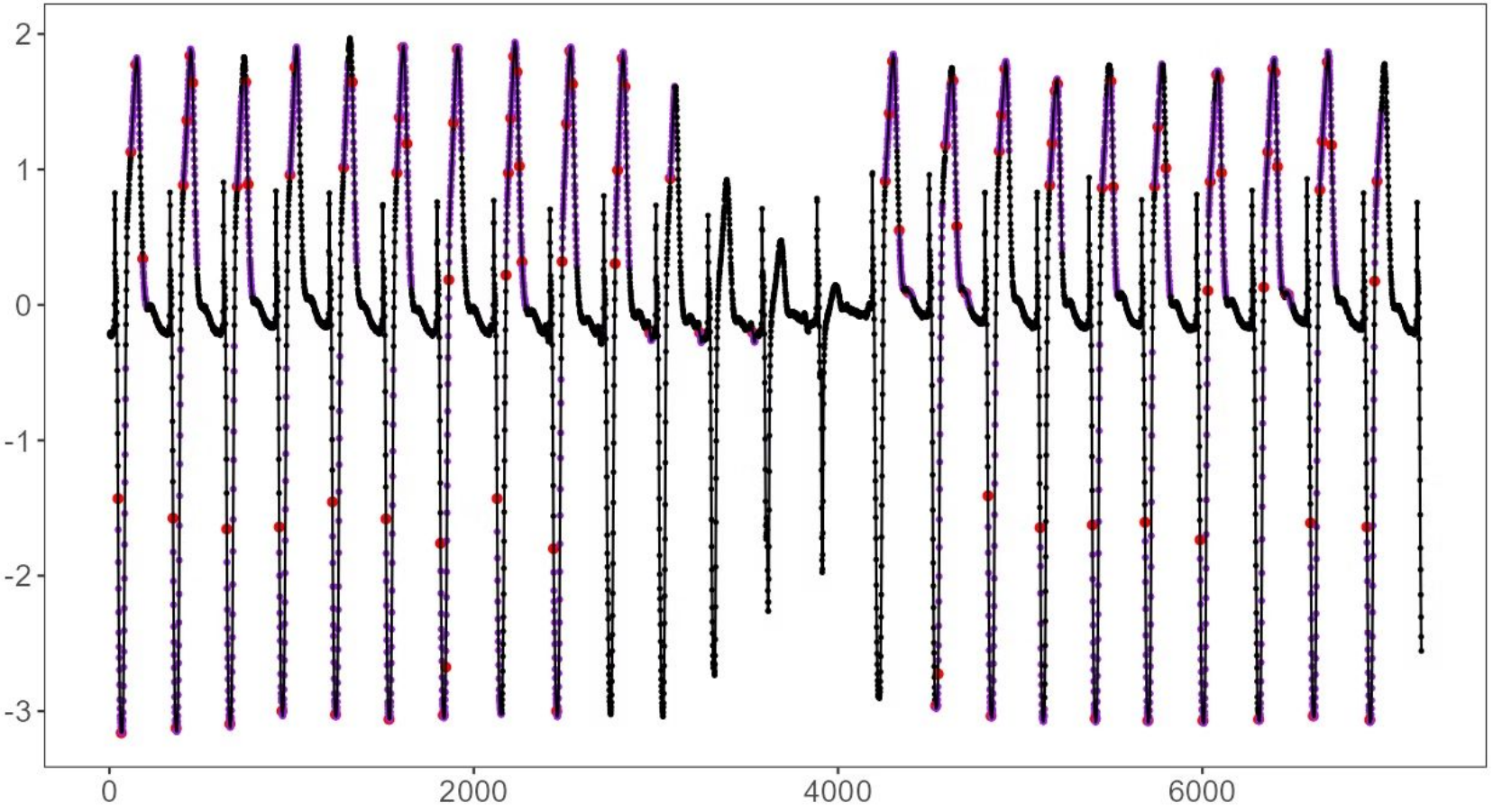
Symbolic grouping (pseudocode)

```
INPUT: SAX words {word_i}, support σ, Hamming h, threshold ε
PRE-CONDITION: words computed for all windows

STEP 1 — index = hash_table()
STEP 2 — For each (i, word_i):
   index[word_i].add(i)
STEP 3 — candidates = {}
STEP 4 — For each (key_a, group_a) in index:
   For each (key_b, group_b) in index:
      If HDist(key_a, key_b) ≤ h:
         candidates.union(group_a ∪ group_b)
STEP 5 — For each group in candidates with |group| ≥ σ:
   Numerically verify pairs in group with dist ≤ ε

OUTPUT: confirmed motifs
POST-CONDITION: false positives eliminated; false negatives possible
```

Motifs detected with SAX



Index-Based Discovery finds motifs in regular heartbeats in the ECG.

Random projection: intuition and algorithm

- Projection reduces the dimension considered in comparison.
- Similar subsequences tend to collide in the same random projections.
- A collision matrix accumulates probabilistic evidence of similarity.
- Motifs are selected among candidates with high support in the matrix.

*Parameters: **n'** = SAX word size; **r** = number of projected positions; **iter** = iterations; **σ** = minimum support*

```
INPUT: series X, word size n', positions r, iterations iter, support σ
PRE-CONDITION: X normalized; SAX words computed

STEP 1 — Compute SAX words for all windows of X
STEP 2 — Collisions = n × n matrix initialized with zeros
STEP 3 — For t from 1 to iter:
   proj = choose r random positions from [1..n']
   bucket = {}
   For each window w_i:
      key = project(word_i, proj)
      bucket[key].add(i)
   For each group in bucket:
      For each pair (i, j) in group, |i-j| > ez:
         Collisions[i][j] += 1
STEP 4 — Candidates = pairs with Collisions[i][j] ≥ σ
STEP 5 — Numerically verify candidates

OUTPUT: motifs confirmed by collision and verification
POST-CONDITION: pairs with high collision have high
                probability of being a motif
```

Random projection: probabilistic analysis

- The collision probability of two similar words increases with the number of iterations.
- Subsequences with small Hamming distance collide with high probability.
- Distant subsequences rarely collide — the method has a low false positive rate.
- The final numerical verification eliminates the remaining false positives.

$$P(\text{collision}) = \left(\frac{\alpha - HDist}{\alpha}\right)^r$$

r = projected positions; *α* = alphabet size; *HDist* = Hamming distance

Interpreting the collision matrix

High collision

Pair (i, j) with high Collisions[i][j] → strong motif candidate. Verify numerically.

Low collision

Pair (i, j) with low Collisions[i][j] → likely not a motif. Discard without verification.

Minimum support σ

Threshold on the collision matrix. Controls the trade-off between recall and verification cost.

Early-abandon and lower bounds

- Early-abandon stops the calculation when the partial error already exceeds ε.
- Saves time by avoiding completing comparisons that cannot become a match.
- Lower bounds estimate a lower limit of the actual distance without computing the full distance.
- If the lower bound already exceeds ε, the pair is discarded at no additional cost.
- LB_Keogh is the classic lower bound for DTW: uses an envelope of width w.

$$LB_Keogh(q, q') \leq DTW(q, q')$$

w = Sakoe-Chiba band width; ε = distance threshold

LB_Keogh (pseudocode)

```
INPUT: sequence q, envelope (U, L) of q', threshold ε
PRE-CONDITION: U[t] = max(q'[t-w..t+w]); L[t] = min(q'[t-w..t+w])

STEP 1 — lb = 0
STEP 2 — For t from 1 to p:
   If q[t] > U[t]:
      lb += (q[t] - U[t])²
   Else if q[t] < L[t]:
      lb += (q[t] - L[t])²
   If lb > ε²:
      Return ∞   (early pruning)
STEP 3 — Return sqrt(lb)

OUTPUT: lower bound of DTW(q, q') or ∞ if it exceeds ε
POST-CONDITION: LB_Keogh ≤ actual DTW; never overestimates
```

Matrix Profile: intuition and algorithm

- For each subsequence, stores the distance to its most similar subsequence (nearest neighbor).
- Vector **M** stores the smallest distance; index **I** stores the neighbor's position.
- In self-join, an **exclusion zone** avoids trivial matches near the diagonal.
- Low distances → **motifs**; high distances → **discords**.

$$M[i] = \min_{j,\ |i-j|>ez} z\text{-}norm\text{-}dist(w_i, w_j)$$

p = window size; *ez* = exclusion zone (default p/2); *n* = series length

```
INPUT: time series X of length n, window size p
PRE-CONDITION: n > 2p; ez = floor(p/2)

STEP 1 — W = extract all windows of X of size p
STEP 2 — M = vector of size (n-p+1) initialized with ∞
         I = vector of size (n-p+1) initialized with -1
STEP 3 — For each pair (i, j), |i - j| > ez:
   d = z-norm-dist(W[i], W[j])
   If d < M[i]: M[i] = d; I[i] = j
   If d < M[j]: M[j] = d; I[j] = i
STEP 4 — Return (M, I)

OUTPUT: Matrix Profile M and index I
POST-CONDITION: M[i] = distance to nearest neighbor of W[i]
                I[i] = position of that neighbor
```

Matrix Profile: Formal Definition

- M is a vector of size (n - p + 1).
- M[i] = z-normalized distance from W[i] to its nearest neighbor (excluding the exclusion zone).
- I[i] = position of the nearest neighbor of W[i].
- In self-join, the exclusion zone avoids trivial matches: |i - j| ≤ ez.
- In AB-join, compares two distinct series X and Y.

$$M[i] = \min_{j:\,|i-j|>ez} \|z(W_i) - z(W_j)\|_2$$

z(·) = z-normalization; *ez* = floor(p/2); *p* = window size

Properties

Completeness

Contains enough information to find all motifs and discords of order k without recomputing distances.

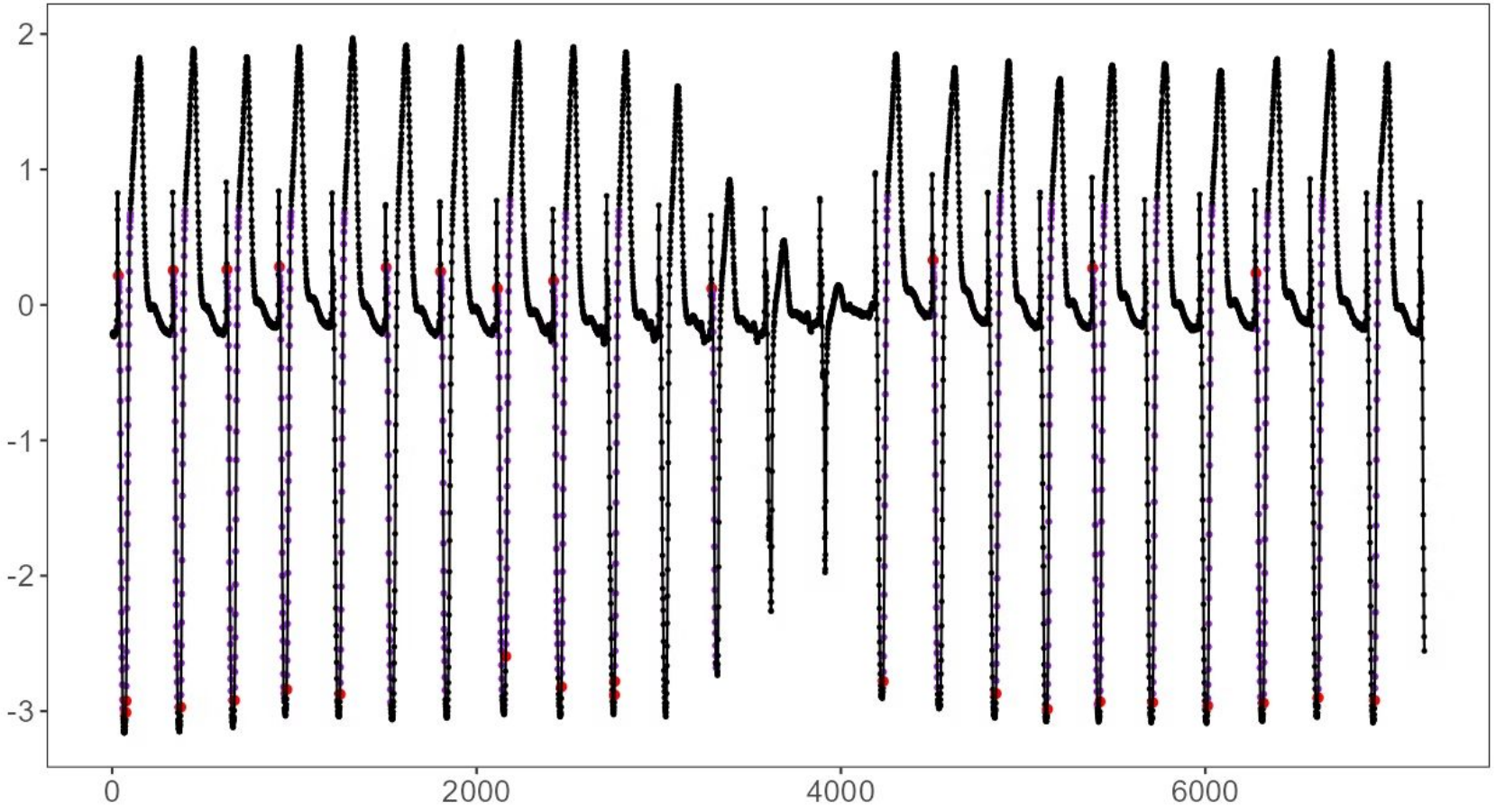
Incrementality

Can be updated as new data arrives (SWAMP for streaming).

Generality

Works with any distance measure that supports z-normalization. Extensible to multiple dimensions (mSTAMP).

Motifs detected with Matrix Profile



Matrix Profile with STAMP also finds motifs in regular heartbeats in ECG data.

Matrix Profile also reveals discords

- The same vector **M** that reveals motifs also points to isolated subsequences.
- This happens because **M** measures the distance to the nearest neighbor.
- If even the nearest neighbor is far away, the subsequence is a discord candidate.
- Motifs and discords are opposite readings of the same vector **M**.

min(M) → Motif

Subsequence with the closest neighbor. Recurring pattern.

max(M) → Discord

Most isolated subsequence. Anomaly candidate.

> "A single pass through the Matrix Profile reveals both the most common and the rarest patterns in the series."

- There is no need to run two separate algorithms.
- The search order (motifs first, discords after) can be adjusted according to the objective.
- Order-*k* discords are found iteratively, excluding regions already selected.

Smallest values: motifs

- A low value in M[i] indicates that a subsequence very similar to W[i] exists.
- The order-1 motif is the pair (i, j) with the smallest value in M.
- Order-k motifs are found iteratively, excluding already selected regions.
- Additional occurrences can be searched from these pairs.

$$motif_1 = \arg\min_i M[i]$$

ez = exclusion zone; k = number of desired motifs

```
INPUT: Matrix Profile M, index I, exclusion zone ez, k motifs
PRE-CONDITION: M and I computed; ez = floor(p/2)

STEP 1 — M_temp = copy of M
STEP 2 — motifs = []
STEP 3 — For t from 1 to k:
   i* = argmin(M_temp)
   j* = I[i*]
   motifs.add((i*, j*))
   For pos in [i* - ez .. i* + ez]:
      M_temp[pos] = +∞   (exclude region)
   For pos in [j* - ez .. j* + ez]:
      M_temp[pos] = +∞   (exclude region)
STEP 4 — Return motifs

OUTPUT: list of k motif pairs in ascending order of distance
POST-CONDITION: each pair is the nearest motif in the remaining region
```

Largest values: discords

Discords via Matrix Profile

- A high value in M indicates that the subsequence has no close neighbors.
- The order-1 discord is the subsequence with the highest value in M.
- Order-k discords are found iteratively, excluding already selected regions.
- This reading connects motif discovery with interval anomaly detection.

$$discord_1 = \arg\max_i M[i]$$

ez = exclusion zone; *k* = number of desired discords

```
INPUT: Matrix Profile M, index I, exclusion zone ez, k discords
PRE-CONDITION: M and I computed; ez = floor(p/2)

STEP 1 — M_temp = copy of M
STEP 2 — discords = []
STEP 3 — For t from 1 to k:
   i* = argmax(M_temp)
   discords.add(i*)
   For j in [i* - ez .. i* + ez]:
      M_temp[j] = -∞   (exclude region)
   For j such that I[j] ∈ [i* - ez .. i* + ez]:
      recompute M_temp[j] without i*
STEP 4 — Return discords

OUTPUT: list of k discord positions in decreasing order of isolation
POST-CONDITION: each discord is the most isolated subsequence in the
remaining region
```

Connection between events

Motifs

Recurrence — subsequences that repeat with distance ≤ ε. They represent typical states or behaviors of the series.

Discords

Isolation — subsequences with no close neighbors (high value in M). They represent anomalies or rare events.

Interval events

Both depend on subsequence, distance, and temporal scale. They form the basis for event analysis in time series.

> Matrix Profile unifies motif and discord discovery in a single structure.

Multivariate Motifs

Can be addressed through dimensionality reduction, combined discoveries, or joint search.

Dimensionality Reduction

PCA, autoencoders, or projections reduce dimensions before searching. May lose relationships between variables if not handled carefully.

Combined Discoveries

Motifs are discovered per dimension and then combined by temporal co-occurrence. Simple, but ignores cross-correlations.

Joint Search (mSTAMP)

Extends Matrix Profile to multiple dimensions simultaneously. Captures relationships between variables without exploding the search space.

> In many cases, multivariate motifs emerge from the interaction between variables — not from each one in isolation.

Variable-size motifs

- Many methods assume a fixed size p — but the actual pattern size may be unknown.
- Searching across multiple values of p is costly: **$O(n^2 \cdot |P|)$** for a set P of sizes.
- **PMP** (Pan Matrix Profile) computes the Matrix Profile for all sizes at once.
- **VALMOD** uses hierarchical search to efficiently find variable-size motifs.

P = set of candidate sizes; $p$min, $p$max = search range

Strategies

Define range [p_min, p_max]

Establish the minimum and maximum bounds of the candidate sizes to investigate.

Compute Matrix Profile for each p

Use PMP or VALMOD to cover the full size range efficiently.

Select optimal motifs

Choose motifs with the lowest distance normalized by size p.

Motif Ranking and Occurrences

Not every recurring pattern is interesting. Motifs can be ranked by different criteria to identify the most relevant ones.

Support

Number of occurrences of the motif. Motifs with more occurrences are more representative of the series.

Similarity

Average distance between occurrences. Motifs with lower internal distance are more cohesive.

Entropy / Rarity

Very frequent motifs can be trivial. Relative rarity may indicate more informative patterns.

Density

Temporal distribution of occurrences. Motifs concentrated in specific regions may indicate regimes or states.

> Nearly constant or trivially similar patterns should be discarded even with high support.

Big data & streaming

Challenges

- Long series make comparison and storage challenging.
- Streaming requires incremental updates to the Matrix Profile.
- Limited memory prevents keeping the entire series in RAM.
- Parallelism and distributed computing are necessary for scale.

Solutions

iSAX

Hierarchical index for long series stored on disk.

SWAMP

Incremental Matrix Profile update for streaming.

mSTAMP

Multivariate Matrix Profile with parallelism support.

Disk-aware

Algorithms that minimize disk access for series that do not fit in memory.

New Structures and Methods

STAMP, STOMP, SCRIMP, PMP and VALMOD

STAMP

Calculates Matrix Profile via random pairs; supports anytime mode.

STOMP

Exploits locality between adjacent columns to reduce cost; $O(n^2)$ with smaller constant.

SCRIMP

Samples diagonals of the distance matrix; anytime and parallelizable.

PMP

Calculates Matrix Profile for multiple window sizes simultaneously.

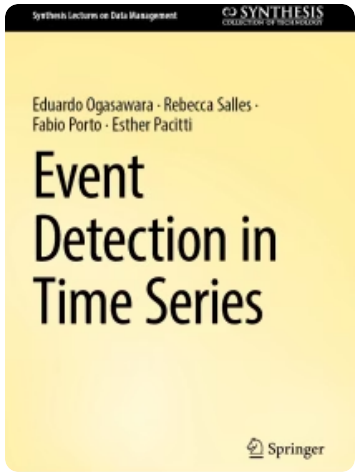
VALMOD

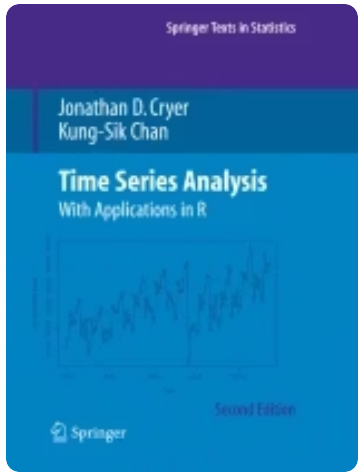
Searches for variable-length motifs without fixing p a priori.

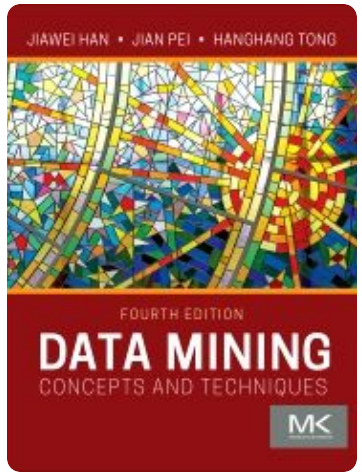
Algorithm Comparison

Algorithm	Strategy	Anytime?	Variable Size?
STAMP	Random pairs	Yes	No
STOMP	Local diagonals	No	No
SCRIMP	Sampled diagonals	Yes	No
PMP	Multiple p	Yes	Yes (via aggregation)
VALMOD	Hierarchical search	No	Yes

Bibliographic References







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4th ed., 2022.